

UNIT 1: JAVA FUNDAMENTALS

- 1.1 Java History and Java Architecture
- 1.2 Java Program Structure
- 1.3 Command Line Arguments
- 1.4 Data Types and Variables
- 1.5 Flow Control Statements
- 1.6 Arrays

1.1 Java History and Java Architecture

Java History

Java is a high-level, class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. It was created by **James Gosling** and his team at **Sun Microsystems** in 1991 as part of a project initially called **Oak**, intended for use in embedded systems. The language was later renamed **Java** and officially released in **1995**.

Java quickly gained popularity due to its platform independence and robust features. In 2010, **Oracle Corporation** acquired Sun Microsystems, and since then, Oracle has maintained and developed the Java platform.

Key Milestones in Java Development:

- **1995:** Java 1.0 released – Introduced the "Write Once, Run Anywhere" concept.
- **1998:** Java 2 introduced (J2SE 1.2) – Enhanced APIs and performance.
- **2004:** Java 5 – Introduced Generics, enhanced for loop, annotations, etc.
- **2011 and beyond:** Java moves to a faster release cycle.
- **Recent LTS versions:** Java 8 (2014), Java 11 (2018), Java 17 (2021), Java 21 (2023).

Features of Java:

1. Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand. Java syntax is based on C++ (so easier for programmers to learn it after C++). Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc. There is an Automatic Garbage Collection in Java, so there is no need to remove unreferenced objects.

2. Object-Oriented

Java is an object-oriented programming language. Everything in Java is an object.

3. Platform independent

Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs.

4. Secured

Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- No explicit pointer
- Java Programs run inside a virtual machine sandbox

5. Robust

The English meaning of Robust is strong. Java is robust because:

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

6. High Performance

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

7. Multithreaded

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

8. Distributed

Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

9. Dynamic

Java is a dynamic language. It supports the dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

10. Portable

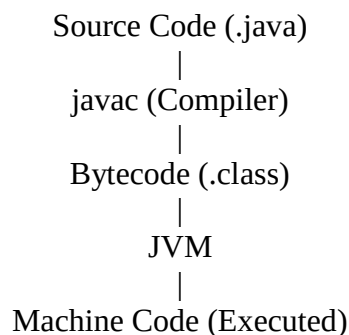
Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

Java Architecture

Java programs go through a series of steps before execution. The architecture includes several components:

1. **Source Code (.java):** The human-readable code written by developers.
2. **Java Compiler (javac):** Translates the source code into **bytecode (.class)** – an intermediate, platform-independent format.
3. **Bytecode:** Cannot be directly understood by the operating system. It is interpreted or compiled by the JVM.
4. **Java Virtual Machine (JVM):** Executes the bytecode on the target machine. The JVM is platform-specific.

Diagram: Java Architecture



1.2 Java Program Structure

Understanding the structure of a basic Java program is essential. Every Java program must have at least one class, and the program's entry point is the `main()` method.

Example Java Program:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Explanation:

- `public class HelloWorld`: Defines a class named HelloWorld. Class names must match the filename.
- `public static void main(String[] args)`: This is the main method, which is the entry point for the program. Without this method, the program cannot run.
- `System.out.println("Hello, World!")`: This line prints the message to the console.

Structure Components:

- **Package Declaration** (optional): Defines the package the class belongs to.
 - **Import Statements** (optional): Imports other Java packages/classes.
 - **Class Definition**: Blueprint of the program.
 - **Main Method**: Entry point for execution.
-

1.3 Command Line Arguments

Java allows users to pass arguments to the main method at runtime through the command line. These arguments are stored as **strings in the array**.

Example Program:

```
public class CommandLineArgs {  
    public static void main(String[] args) {  
        System.out.println("Number of arguments: " + args.length);  
        for (int i = 0; i < args.length; i++) {  
            System.out.println("Argument " + i + ": " + args[i]);  
        }  
    }  
}
```

How to Run:

1. Compile: `javac CommandLineArgs.java`
2. Execute: `java CommandLineArgs Hello World`

Output:

```
Number of arguments: 2  
Argument 0: Hello  
Argument 1: World
```

Command line arguments are especially useful when passing runtime values to programs.

1.4 Data Types and Variables

Java Data Types

Java is a statically-typed language, meaning every variable must be declared with a data type. Java has two categories:

1. Primitive Data Types:

- **Integer Types:** byte (1 byte), short (2 bytes), int (4 bytes), long (8 bytes)
- **Floating Point Types:** float (4 bytes), double (8 bytes)
- **Character Type:** char (2 bytes, Unicode)
- **Boolean Type:** boolean (true or false)

2. Non-Primitive Data Types:

- **Objects** (instances of classes)
- **Arrays**
- **Strings**

Variables in Java

A variable is a container for storing data values. Each variable must be declared with a data type. Java supports **local variables**, **instance variables**, and **static variables**, depending on where they are declared.

//DataTypeExample.java

```
public class DataTypeExample {  
    public static void main(String[] args) {  
        byte age = 25;  
        short year = 2025;  
        int salary = 50000;  
        long population = 79000000000L;  
  
        float pi = 3.14f;  
        double g = 9.81;  
  
        char grade = 'A';  
        boolean isPass = true;  
        String name="cocsit";  
        int[ ] a={10,20,30};  
  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

```

System.out.println("Year: " + year);
System.out.println("Salary: " + salary);
System.out.println("Population: " + population);
System.out.println("PI: " + pi);
System.out.println("Gravity: " + g);
System.out.println("Grade: " + grade);
System.out.println("Passed: " + isPass);

    for(int j=0; j<a.length; j++){
        System.out.println("A: " + a[j]);
    }
}
}

```

//ReadFromKeyboard.java

```

import java.util.Scanner;
public class ReadFromKeyboard {
    public static void main(String[] args) {
        // Create Scanner object
        Scanner scanner = new Scanner(System.in);

        // Reading a string
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        // Reading an integer
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        // Reading a double
        System.out.print("Enter your height in meters: ");
        double height = scanner.nextDouble();

        // Output the values
        System.out.println("\n--- User Details ---");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Height: " + height + " meters");

        // Close the scanner
        scanner.close();
    }
}

```

Control statements in Java

Java compiler executes the java code from top to bottom. The statements are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of java code. Such statements are called control statements.

Java provides three types of control statements.

- Decision Making statements
- Looping statements
- Jumping statements

Decision-Making statements:

Decision-making statements evaluate the Boolean expression and control the program flow depending upon the condition result. There are two types of decision-making statements in java, i.e., if statement and switch statement.

If Statement:

In Java, if statement is used to evaluate a condition. The control of the program is diverted depending upon the condition result which is a Boolean value, either true or false. In java, there are four types of if-statements given below.

- if statement
- if-else statement
- else-if statement
- Nested if-statement

Let's understand the if-statements one by one.

1. If statement:

This is the most basic statement among all control flow statements in java. It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.

Syntax of if statement is given below.

```
if(<condition>) {  
  
    //block of code  
  
}
```

Consider the following example in which we have used the if statement in the java code.

```

public class Student {
    public static void main(String[] args) {
        int x = 10;
        int y = 12;
        if(x+y > 20) {
            System.out.println("x + y is greater than 20");
        }
    }
}

```

Output:

x + y is greater than 20

2. if-else statement

The if-else statement is an extension to the if-statement, which uses another block of code, I.e., else block. The else block is executed if the condition of the if-block is evaluated as false.

Consider the following example.

```

public class Ss{
    public static void main(String[] args) {
        int x = 10;
        int y = 12;
        if(x+y < 10) {
            System.out.println("x + y is less than 10");
        } else {
            System.out.println("x + y is greater than 20");
        }
    }
}

```

Output:

x + y is greater than 20

3. Else-if statement

The else-if statement contains the if-statement followed by multiple else-if statements. In other words, we can say that it is the chain of if-else statements that create a decision tree where the program may enter any block of code. We can also define an else statement at the end of the chain.

Consider the following example.

```

public class city {
    public static void main(String[] args) {
        String city = "Delhi";
    }
}

```



```

if(city == "Meerut") {
    System.out.println("city is meerut");
} else if (city == "Noida") {
    System.out.println("city is noida");
} else if(city == "Agra") {
    System.out.println("city is agra");
} else {
    System.out.println(city);
}
}
}

```

Output:

Delhi

4. Nested if-statement

In nested if-statements, the if statement contains multiple if-else statements as a separate block of code. Consider the following example.

```

public class Student {
    public static void main(String[] args) {
        String address = "Delhi, India";
        if(address.endsWith("India")) {
            if(address.contains("Meerut")) {
                System.out.println("Your city is meerut");
            }
            else if(address.contains("Noida")) {
                System.out.println("Your city is noida");
            }
            else {
                System.out.println(address.split(",")[0]);
            }
        }
        else {
            System.out.println("You are not living in india");
        }
    }
}

```

Output:

Delhi

Switch Statement:

In Java, Switch statements are similar to if-else-if statements. The switch statement enables us to check the variable for the range of values defined for multiple case statements. The switch statement is easier to use instead of if-else-if statements. It also enhances the readability of the program. The syntax to use the switch statement is given below.

```

switch <variable> {

Case <option 1>:

//block of statements

..

..

..

Case <option n>:

//block of statements

Default:

//block of statements

}

```

Consider the following example to understand the flow of the switch statement.

```

import java.util.*;
public class S implements Cloneable {
public static void main(String[] args) {
Scanner sc=new Scanner(System.in);
System.out.println("1. Addition");
System.out.println("2. Subtraction");
System.out.println("3. Multiplication");
System.out.println("4. Division");

System.out.println("Enter your choice : ");
int ch=sc.nextInt();
int num1=30;
int num2=56;
switch (ch){
case 1:
System.out.println("Addition is "+(num1+num2));
break;
case 2:
System.out.println("Subtraction is "+(num1-num2));
break;
case 3:
System.out.println("Multiplication is "+(num1*num2));
break;
case 4:
System.out.println("Division is "+(num1/num2));
break;

```

```
default:
System.out.println("Invalid choice");
} } }
```

Output:

```
D:\>javac S.java
```

```
D:\>java S
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice :
1
Addition is 86
```

```
D:\>java S
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice :
2
Subtraction is -26
```

```
D:\>java S
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice :
3
Multiplication is 1680
```

```
D:\>java S
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice :
4
Division is 0
```

```
D:\>java S
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice :
9
Invalid choice
```

While using switch statements, we must notice that the case expression will be of the same type as the variable. However, it will also be a constant value. The switch permits only int, string, and Enum type variables to be used.

Loop Statements

In programming, sometimes we need to execute the block of code repeatedly while some condition evaluates to true. However, loop statements are used to execute the set of instructions in a repeated order. The execution of the set of instructions depends upon a particular condition.

In Java, we have three types of loops that execute similarly. However, there are differences in their syntax and condition checking time.

1. for loop
2. while loop
3. do-while loop

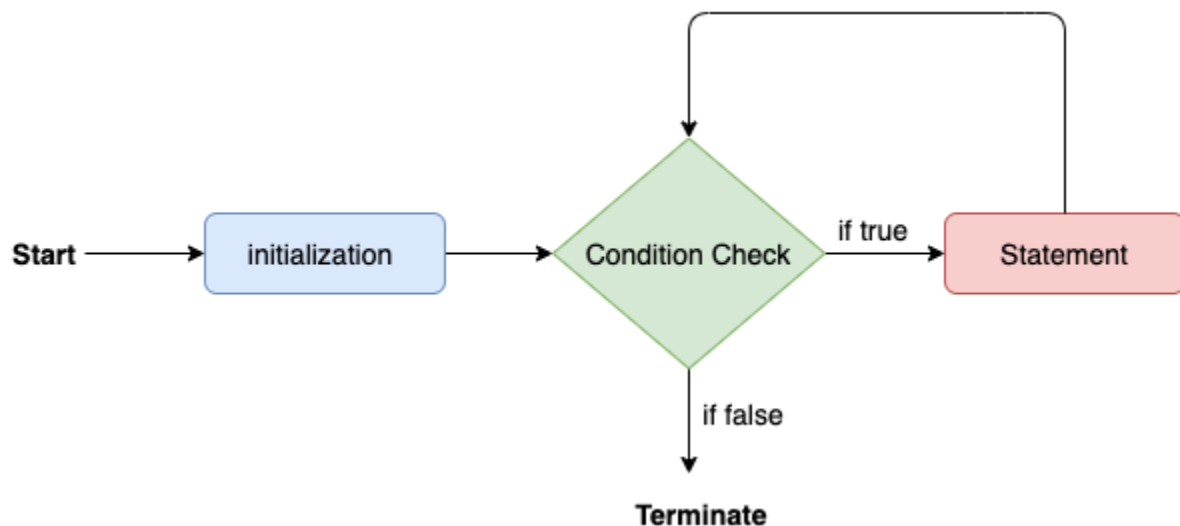
Let's understand the loop statements one by one.

Java for loop

In java, for loop is similar to C and C ++. It enables us to initialize the loop variable, check the condition, and increment/decrement in a single line of code. The syntax of for loop is given below.

1. for(<initialization>, <condition>, <increment/decrement>) {
2. //block of statements
3. }

The flow chart for the for-loop is given below.



Consider the following example to understand the proper functioning of the for loop in java.

- ```
public class ForEx {
 public static void main(String[] args) {
 //Code of Java for loop
 for(int i=1;i<=10;i++){
 System.out.print(i);
 }
 }
}
```

o/p- 1 2 3 4 ....10
- ```
public class NestedForEx{  
    public static void main(String[] args) {  
        //loop of i  
        for(int i=1;i<=3;i++){  
            //loop of j  
            for(int j=1;j<=3;j++){  
                System.out.println(i+" "+j);  
            }  
        }  
    }  
}
```
- ```
public class PyramidEx {
 public static void main(String[] args) {
 for(int i=1;i<=5;i++){
 for(int j=1;j<=i;j++){
 System.out.print("* ");
 }
 System.out.println();//new line
 }
 }
}
```

## Java for-each loop

Java provides an enhanced for loop to traverse the data structures like array or collection. In for-each loop, we don't need to update the loop variable. The syntax to use for-each loop in java is given below.

```
for (type var : collection){

 //statements

}
```

Consider the following example to understand the functioning of the for-each loop in java.

```
class foreachEX{

 public static void main(String[] args) {

 String[] names = {"Java", "C", "C++", "Python", "JavaScript"};

 System.out.println("Printing the contents of an array :\n");

 for(String n:names) {

 System.out.println(n);

 } } }
```

### Output:

Printing the content of the array names:

```
Java
C
C++
Python
JavaScript
```

## Java While loop

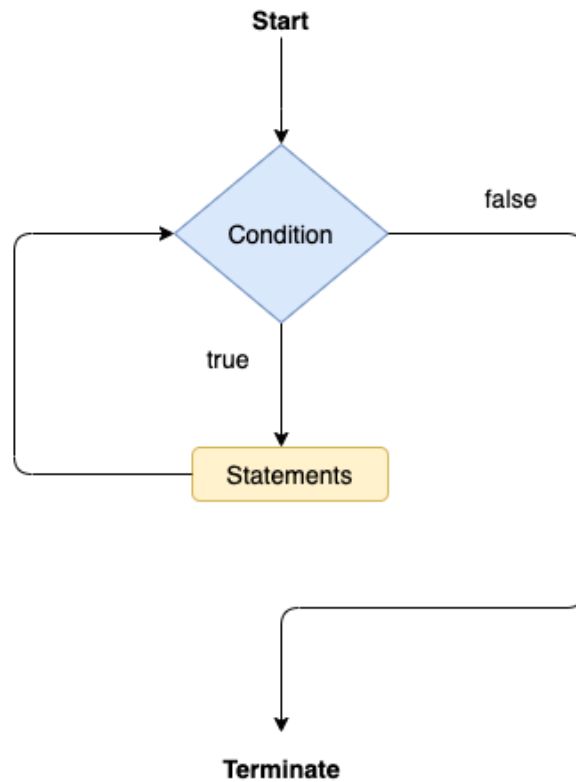
The while loop is also used to repeat the statements multiple times. It is also known as the entry-controlled loop since the condition is checked at the start of the loop. If the condition is true, then the loop body will be executed; otherwise, the statements after the loop will be executed.

The syntax of the while loop is given below.

```
1. While(<condition>){
```

2. //loop statements
3. }

The flow chart for the while loop is given in the following image.



Consider the following example.

```
public class whileEx {
 public static void main(String[] args) {
 int i=10;
 while(i>=1){
 System.out.println(i);
 i-- ;
 }
 }
}
```

**Output:**

```
10
9
8
7
... 1
```

## Java do-while loop

The do-while loop checks the condition at the end of the loop after executing the loop statements. However, it is recommended to use the do-while loop if we don't know the condition in advance, and we need the loop to execute at least once.

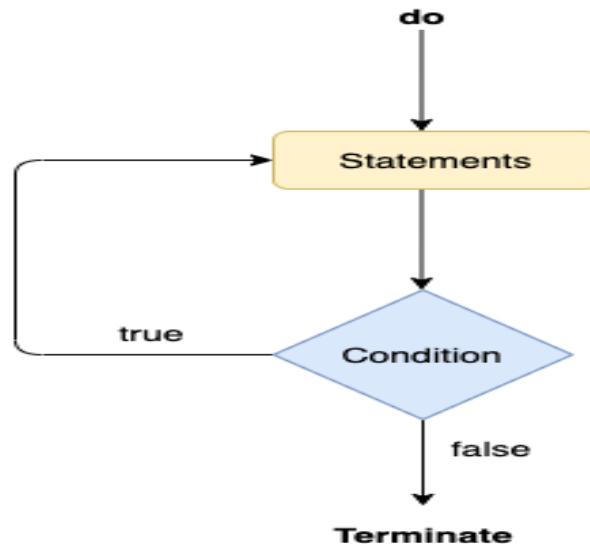
It is also known as the exit-controlled loop since the condition is not checked in advance. The syntax of the do-while loop is given below.

```
do

{ //statements

} while (<Condition>);
```

The flow chart of the do-while loop is given in the following image.



Consider the following example to understand the functioning of the do-while loop in java.

```
public class DoWhileEx {
 public static void main(String[] args) {
 int i=1;
 do{
 System.out.println(i);
 i++;
 } while (i<10);
 }
}
```

**Output:** 1 2 3 4 ...10 (new line)



## Jump Statements

Jump statements are used to transfer the control of the program to the specific statements. In other words, jump statements transfer the execution control to the other part of the program. There are two types of jump statements in java, i.e., break and continue.

### Java break statement

As the name suggests, the break statement is used to break the current flow of the program and transfer the control to the next statement outside the current flow. It is used to break the loop and switch statement. However, it breaks only the inner loop in case of the nested loop.

When a break statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.

The break statement cannot be used independently in the java program, i.e., it can only be written inside the loop or switch statement.

Consider the following example in which we have used the break statement with the for loop.

- ```
public class BrEx {  
    public static void main(String[] args) {  
        for(int i = 0; i<= 10; i++) {  
            if(i==6) {  
                break; }  
            System.out.println(i);  
        } } }
```

Output:

```
0  
1  
2  
3  
4  
5
```

Java continue statement

Unlike break statement, the continue statement doesn't break the loop, whereas, it skips the specific part of the loop and jumps to the next iteration of the loop immediately.

The Java continue statement is used to continue the loop. It continues the current flow of the program and skips the remaining code at the specified condition. In case of an inner loop, it continues the inner loop only.

Consider the following example to understand the functioning of the continue statement in java.

- ```
public class ContinueExample {
 public static void main(String[] args) {
 //for loop
 for(int i=1;i<=10;i++){
 if(i==5){
 //using continue statement
 continue;//it will skip the rest statement
 }
 System.out.println(i);
 }
 }
}
```

**Output:**

```
1
2
3
4
6
7
8
9
10
```

## 1.6 Arrays

**Arrays in Java** are one of the most fundamental data structures that allow us to store multiple values of the same type in a single variable. They are useful for storing and managing collections of data. Arrays in Java are objects, which makes them work differently from arrays in C/C++ in terms of memory management. For **primitive arrays**, **elements** are stored in a contiguous memory location, For **non-primitive arrays**, **references** are stored at contiguous locations, but the actual objects may be at different locations in memory.

**Key features of Arrays:**

- **Contiguous Memory Allocation (for Primitives):** Java array elements are stored in continuous memory locations, which means that the elements are placed next to each other in memory.
- **Zero-based Indexing:** The first element of the array is at index 0.
- **Fixed Length:** Once an array is created, its size is fixed and cannot be changed.
- **Can Store Primitives & Objects:** Java arrays can hold both primitive types (like int, char, boolean, etc.) and objects (like String, Integer, etc.)

# Creating, Initializing, and Accessing an Arrays in Java

For understanding the array we need to understand how it actually works. To understand this follow the flow mentioned below:

- Declare
- Initialize
- Access

## 1. Declaring an Array

The general form of array declaration is

**Method 1:**

*type var-name[];*

**Method 2:**

*type[] var-name;*

The element type determines the data type of each element that comprises the array. Like an array of integers, we can also create an array of other primitive data types like char, float, double, etc., or user-defined data types (objects of a class).

**Note:** It is just how we can create is an array variable, **no actual array exists**. It merely tells the compiler that this variable (int Array) will hold an array of the integer type.

Now, Let us provide memory storage to this created array.

## 2. Initialization an Array in Java

When an array is declared, only a reference of an array is created. The general form of *new* as it applies to one-dimensional arrays appears as follows:

*var-name = new type [size];*

Here, *type* specifies the type of data being allocated, *size* determines the number of elements in the array, and *var-name* is the name of the array variable that is linked to the array. To use *new* to allocate an array, **you must specify the type and number of elements to allocate**.

**Example:**

*// declaring array*

*int intArray[];*

*// allocating memory to array*

*intArray = new int[20];*

*// combining both statements in one*

*int[] intArray = new int[20];*

## 3. Accessing Java Array Elements using for Loop

Now, we have created an Array with or without the values stored in it. Access becomes an important part to operate over the values mentioned within the array indexes using the points mentioned below:

- Each element in the array is accessed via its index.
- The index begins with 0 and ends at (total array size)-1.
- All the elements of array can be accessed using Java for Loop.

Let us check the syntax of basic for loop to traverse an array:

*// Accessing the elements of the specified array*

*for (int i = 0; i < arr.length; i++)*

*System.out.println("Element at index " + i + " : " + arr[i]);*

Example –

```
class GFG {
 public static void main(String[] args)
 {
 // declares an Array of integers.
 int[] arr;

 // allocating memory for 5 integers.
 arr = new int[5];

 // initialize the elements of the array
 // first to last(fifth) element
 arr[0] = 10;
 arr[1] = 20;
 arr[2] = 30;
 arr[3] = 40;
 arr[4] = 50;

 // accessing the elements of the specified array
 for (int i = 0; i < arr.length; i++)
 System.out.println("Element at index "
 + i + " : " + arr[i]);
 }
}
```

o/p -

```
Element at index 0 : 10
Element at index 1 : 20
Element at index 2 : 30
Element at index 3 : 40
Element at index 4 : 50
```

\*\*\*